

Chapter 1

Inheritance

Inheritance is one of the cornerstones of object-oriented programming because it allows the creation of hierarchical classifications. Using inheritance, you can create a general class that defines traits common to a set of related items. This class can then be inherited by other. In the terminology of Java, a class that is inherited is called a superclass. The class that does the inheriting is called a subclass.

Therefore, a subclass is a specialized version of a superclass. It inherits all of the members defined by the superclass and adds its own, unique elements.

INHERITANCE BASICS

To inherit a class, you simply incorporate the definition of one class into another by using the “**extends**” keyword. To see how, let’s begin with a short example.

The following program creates a superclass called “A” and a subclass called “B”.

// A simple example of inheritance.

```
// Create a superclass.
class A {
    int i, j;

    void showij() {
        System.out.println("i and j: " + i + " " + j);
    }
}

// Create a subclass by extending class A.
class B extends A {
    int k;

    void showk() {
        System.out.println("k: " + k);
    }

    void sum() {
        System.out.println("i+j+k: " + (i+j+k));
    }
}
```

```

class SimpleInheritance {
    public static void main(String args []) {
        A superOb = new A();
        B subOb = new B();

        // The superclass may be used by itself.
        superOb.i = 10;
        superOb.j = 20;
        System.out.println("Contents of superOb: ");
        superOb.showij();
        System.out.println();

        /* The subclass has access to all public members of
           its superclass. */
        subOb.i = 7;
        subOb.j = 8;
        subOb.k = 9;
        System.out.println("Contents of subOb: ");
        subOb.showij();
        subOb.showk();
        System.out.println();

        System.out.println("Sum of i, j and k in subOb:");
        subOb.sum();
    }
}

```

The output from this program is shown here:

```

Contents of superOb:
i and j: 10 20

Contents of subOb:
i and j: 7 8
k: 9

Sum of i, j and k in subOb:
i+j+k: 24

```

As you can see, the subclass **B** includes all of the members of its superclass, **A**. This is why **subOb** can access **i** and **j** and call **showij()**. Also, inside **sum()**, **i** and **j** can be referred to directly, as if they were part of **B**. Even though **A** is a superclass for **B**, it is also a completely independent, standalone class. Being a superclass for a subclass does not mean that the

superclass cannot be used by itself. Further, a subclass can be a superclass for another subclass.

The general form of a **class** declaration that inherits a superclass is shown here:

```
class subclass-name extends superclass-name {  
    // body of class  
}
```

Member Access and Inheritance

Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as **private**. For example, consider the following simple class hierarchy:

```
/* In a class hierarchy, private members remain  
   private to their class.
```

```
   This program contains an error and will not  
   compile.
```

```
*/
```

```
// Create a superclass.
```

```
class A {  
    int i; // public by default  
    private int j; // private to A  
  
    void setij(int x, int y) {  
        i = x;  
        j = y;  
    }  
}
```

```
// A's j is not accessible here.
```

```
class B extends A {  
    int total;  
  
    void sum() {  
        total = i + j; // ERROR, j is not accessible here  
    }  
}
```

```
class Access {  
    public static void main(String args[]) {  
        B subOb = new B();
```

```

        subOb.setij(10, 12);

        subOb.sum();
        System.out.println("Total is " + subOb.total);
    }
}

```

This program will not compile because the use of **j** inside the **sum()** method of **B** causes an access violation. Since **j** is declared as **private**, it is only accessible by other members of its own class. Subclasses have no access to it.

A More Practical Example

```

// This program uses inheritance to extend Box.
class Box {
    double width;
    double height;
    double depth;

    // construct clone of an object
    Box(Box ob) { // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }

    // constructor used when all dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // constructor used when no dimensions specified
    Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
    }
}

```

```

// constructor used when cube is created
Box(double len) {
    width = height = depth = len;
}

// compute and return volume
double volume() {
    return width * height * depth;
}
}

// Here, Box is extended to include weight.
class BoxWeight extends Box {
    double weight; // weight of box

    // constructor for BoxWeight
    BoxWeight(double w, double h, double d, double m) {
        width = w;
        height = h;
        depth = d;
        weight = m;
    }
}

class DemoBoxWeight {
    public static void main(String args[]) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
        double vol;

        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        System.out.println("Weight of mybox1 is " + mybox1.weight);
        System.out.println();

        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        System.out.println("Weight of mybox2 is " + mybox2.weight);
    }
}

```

The output from this program is shown here:

```
Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3
```

```
Volume of mybox2 is 24.0
Weight of mybox2 is 0.076
```

BoxWeight inherits all of the characteristics of **Box** and adds to them the **weight** component. It is not necessary for **BoxWeight** to re-create all of the features found in **Box**. It can simply extend **Box** to meet its own purposes.

A major advantage of inheritance is that once you have created a superclass that defines the attributes common to a set of objects, it can be used to create any number of more specific subclasses.

For example, the following class inherits **Box** and adds a color attribute:

```
// Here, Box is extended to include color.
class ColorBox extends Box {
    int color; // color of box

    ColorBox(double w, double h, double d, int c) {
        width = w;
        height = h;
        depth = d;
        color = c;
    }
}
```

A Superclass Variable Can Reference a Subclass Object

A reference variable of a superclass can be assigned a reference to any subclass derived from that superclass. For example, consider the following:

```
class RefDemo {
    public static void main(String args[]) {
        BoxWeight weightbox = new BoxWeight(3, 5, 7, 8.37);
        Box plainbox = new Box();
        double vol;

        vol = weightbox.volume();
        System.out.println("Volume of weightbox is " + vol);
        System.out.println("Weight of weightbox is " +
                           weightbox.weight);
        System.out.println();

        // assign BoxWeight reference to Box reference
        plainbox = weightbox;
    }
}
```

```

    vol = plainbox.volume(); // OK, volume() defined in Box
    System.out.println("Volume of plainbox is " + vol);

    /* The following statement is invalid because plainbox
       does not define a weight member. */
    // System.out.println("Weight of plainbox is " + plainbox.weight);
}
}

```

Here, **weightbox** is a reference to **BoxWeight** objects, and **plainbox** is a reference to **Box** objects. Since **BoxWeight** is a subclass of **Box**, it is permissible to assign **plainbox** a reference to the **weightbox** object.

It is important to understand that it is the type of the reference variable—not the type of the object that it refers to—that determines what members can be accessed. That is, when a reference to a subclass object is assigned to a superclass reference variable, you will have access only to those parts of the object defined by the superclass. This is why **plainbox** can't access **weight** even when it refers to a **BoxWeight** object. If you think about it, this makes sense, because the superclass has no knowledge of what a subclass adds to it. This is why the last line of code in the preceding fragment is commented out. It is not possible for a **Box** reference to access the **weight** field, because **Box** does not define one.

USING super

Whenever a subclass needs to refer to its immediate superclass, it can do so by use of the keyword **super**. **super** has two general forms. The first calls the superclass' constructor. The second is used to access a member of the superclass that has been hidden by a member of a subclass. Each use is examined here.

Using super to Call Superclass Constructors

A subclass can call a constructor defined by its superclass by use of the following form of **super**: **super(arg-list)**;

Here, *arg-list* specifies any arguments needed by the constructor in the superclass. **super()** must always be the first statement executed inside a subclass' constructor. To see how **super()** is used, consider this improved version of the

BoxWeight class:

```

// BoxWeight now uses super to initialize its Box attributes.
class BoxWeight extends Box {
    double weight; // weight of box
}

```

```

// initialize width, height, and depth using super()
BoxWeight(double w, double h, double d, double m) {
    super(w, h, d); // call superclass constructor
    weight = m;
}
}

```

Here, **BoxWeight()** calls **super()** with the arguments **w**, **h**, and **d**. This causes the **Box** constructor to be called, which initializes **width**, **height**, and **depth** using these values. **BoxWeight** no longer initializes these values itself. It only needs to initialize the value unique to it: **weight**.

In the preceding example, **super()** was called with three arguments. Since constructors can be overloaded, **super()** can be called using any form defined by the superclass. The constructor executed will be the one that matches the arguments.

// A complete implementation of BoxWeight.

```

class Box {
    private double width;
    private double height;
    private double depth;

    // construct clone of an object
    Box(Box ob) { // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }

    // constructor used when all dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // constructor used when no dimensions specified
    Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
    }
}

```



```

// constructor used when cube is created
Box(double len) {
    width = height = depth = len;
}

// compute and return volume
double volume() {
    return width * height * depth;
}
}

// BoxWeight now fully implements all constructors.
class BoxWeight extends Box {
    double weight; // weight of box

    // construct clone of an object
    BoxWeight(BoxWeight ob) { // pass object to constructor
        super(ob);
        weight = ob.weight;
    }

    // constructor when all parameters are specified
    BoxWeight(double w, double h, double d, double m) {
        super(w, h, d); // call superclass constructor
        weight = m;
    }

    // default constructor
    BoxWeight() {
        super();
        weight = -1;
    }

    // constructor used when cube is created
    BoxWeight(double len, double m) {
        super(len);
        weight = m;
    }
}

```

```

class DemoSuper {
    public static void main(String args[]) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
        BoxWeight mybox3 = new BoxWeight(); // default
        BoxWeight mycube = new BoxWeight(3, 2);
        BoxWeight myclone = new BoxWeight(mybox1);
        double vol;

        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        System.out.println("Weight of mybox1 is " + mybox1.weight);
        System.out.println();

        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        System.out.println("Weight of mybox2 is " + mybox2.weight);
        System.out.println();

        vol = mybox3.volume();
        System.out.println("Volume of mybox3 is " + vol);
        System.out.println("Weight of mybox3 is " + mybox3.weight);
        System.out.println();

        vol = myclone.volume();
        System.out.println("Volume of myclone is " + vol);
        System.out.println("Weight of myclone is " + myclone.weight);
        System.out.println();

        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
        System.out.println("Weight of mycube is " + mycube.weight);
        System.out.println();
    }
}

```

This program generates the following output:

```

Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3

```

```

Volume of mybox2 is 24.0
Weight of mybox2 is 0.076

```

```
Volume of mybox3 is -1.0
Weight of mybox3 is -1.0
```

```
Volume of myclone is 3000.0
Weight of myclone is 34.3
```

```
Volume of mycube is 27.0
Weight of mycube is 2.0
```

Pay special attention to this constructor in **BoxWeight**:

```
// construct clone of an object
BoxWeight(BoxWeight ob) { // pass object to constructor
    super(ob);
    weight = ob.weight;
}
```

Notice that **super()** is passed an object of type **BoxWeight**—not of type **Box**. This still invokes the constructor **Box(Box ob)**. As mentioned earlier, a superclass variable can be used to reference any object derived from that class. Thus, we are able to pass a **BoxWeight** object to the **Box** constructor.

Let's review the key concepts behind **super()**. When a subclass calls **super()**, it is calling the constructor of its immediate superclass. Thus, **super()** always refers to the superclass immediately above the calling class. Also, **super()** must always be the first statement executed inside a subclass constructor.

A Second Use for super

This usage has the following general form:

super.member

Here, *member* can be either a method or an instance variable.

This second form of **super** is most applicable to situations in which member names of a subclass hide members by the same name in the superclass. Consider this simple class hierarchy:

```
// Using super to overcome name hiding.
class A {
    int i;
}
```

```
// Create a subclass by extending class A.
class B extends A {
    int i; // this i hides the i in A
```

```

    B(int a, int b) {
        super.i = a; // i in A
        i = b; // i in B
    }

    void show() {
        System.out.println("i in superclass: " + super.i);
        System.out.println("i in subclass: " + i);
    }
}

class UseSuper {
    public static void main(String args[]) {
        B subOb = new B(1, 2);

        subOb.show();
    }
}

```

This program displays the following:

```
i in superclass: 1
```

```
i in subclass: 2
```

Although the instance variable **i** in **B** hides the **i** in **A**, **super** allows access to the **i** defined in the superclass. As you will see, **super** can also be used to call methods that are hidden by a subclass.

CREATING A MULTILEVEL HIERARCHY

Up to this point, we have been using simple class hierarchies that consist of only a superclass and a subclass. However, you can build hierarchies that contain as many layers of inheritance as you like. As mentioned, it is perfectly acceptable to use a subclass as a superclass of another. For example, given three classes called **A**, **B**, and **C**, **C** can be a subclass of **B**, which is a subclass of **A**. When this type of situation occurs, each subclass inherits all of the traits found in all of its superclasses. In this case, **C** inherits all aspects of **B** and **A**.

```

// Extend BoxWeight to include shipping costs.

// Start with Box.
class Box {
    private double width;
    private double height;
    private double depth;
    // construct clone of an object
    Box(Box ob) { // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }

    // constructor used when all dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // constructor used when no dimensions specified
    Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
    }

    // constructor used when cube is created
    Box(double len) {
        width = height = depth = len;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

```

```

// Add weight.
class BoxWeight extends Box {
    double weight; // weight of box

    // construct clone of an object
    BoxWeight(BoxWeight ob) { // pass object to constructor
        super(ob);
        weight = ob.weight;
    }

    // constructor when all parameters are specified
    BoxWeight(double w, double h, double d, double m) {
        super(w, h, d); // call superclass constructor
        weight = m;
    }

    // default constructor
    BoxWeight() {
        super();
        weight = -1;
    }

    // constructor used when cube is created
    BoxWeight(double len, double m) {
        super(len);
        weight = m;
    }
}

// Add shipping costs.
class Shipment extends BoxWeight {
    double cost;

    // construct clone of an object
    Shipment(Shipment ob) { // pass object to constructor
        super(ob);
        cost = ob.cost;
    }
}

```

```

// constructor when all parameters are specified
Shipment(double w, double h, double d,
          double m, double c) {
    super(w, h, d, m); // call superclass constructor
    cost = c;
}

// default constructor
Shipment() {
    super();
    cost = -1;
}

// constructor used when cube is created
Shipment(double len, double m, double c) {
    super(len, m);
    cost = c;
}
}

class DemoShipment {
    public static void main(String args[]) {
        Shipment shipment1 =
            new Shipment(10, 20, 15, 10, 3.41);
        Shipment shipment2 =
            new Shipment(2, 3, 4, 0.76, 1.28);

        double vol;

        vol = shipment1.volume();
        System.out.println("Volume of shipment1 is " + vol);
        System.out.println("Weight of shipment1 is "
            + shipment1.weight);
        System.out.println("Shipping cost: $" + shipment1.cost);
        System.out.println();
        vol = shipment2.volume();
        System.out.println("Volume of shipment2 is " + vol);
        System.out.println("Weight of shipment2 is "
            + shipment2.weight);
        System.out.println("Shipping cost: $" + shipment2.cost);
    }
}

```

The output of this program is shown here:
Volume of shipment1 is 3000.0
Weight of shipment1 is 10.0
Shipping cost: \$3.41

Volume of shipment2 is 24.0
Weight of shipment2 is 0.76
Shipping cost: \$1.28

Because of inheritance, **Shipment** can make use of the previously defined classes of **Box** and **BoxWeight**, adding only the extra information it needs for its own, specific application. This is part of the value of inheritance; it allows the reuse of code.

This example illustrates one other important point: **super()** always refers to the constructor in the closest superclass. The **super()** in **Shipment** calls the constructor in **BoxWeight**. The **super()** in **BoxWeight** calls the constructor in **Box**.

METHOD OVERRIDING

In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to *override* the method in the superclass. When an overridden method is called from within its subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden. Consider the following:

```
// Method overriding.
class A {
    int i, j;
    A(int a, int b) {
        i = a;
        j = b;
    }

    // display i and j
    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}
```



```

class B extends A {
    int k;

    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }

    // display k - this overrides show() in A
    void show() {
        System.out.println("k: " + k);
    }
}

class Override {
    public static void main(String args[]) {
        B subOb = new B(1, 2, 3);

        subOb.show(); // this calls show() in B
    }
}

```

The output produced by this program is shown here:

k: 3

When **show()** is invoked on an object of type **B**, the version of **show()** defined within **B** is used. That is, the version of **show()** inside **B** overrides the version declared in **A**.

If you wish to access the superclass version of an overridden method, you can do so by using **super**. For example, in this version of **B**, the superclass version of **show()** is invoked within the subclass' version. This allows all instance variables to be displayed.

```

class B extends A {
    int k;

    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }
}

```

```

    void show() {
        super.show(); // this calls A's show()
        System.out.println("k: " + k);
    }
}

```

If you substitute this version of **A** into the previous program, you will see the following output:

i and j: 1 2

k: 3

Here, **super.show()** calls the superclass version of **show()**.

Method overriding occurs *only* when the names and the type signatures of the two methods are identical. If they are not, then the two methods are simply overloaded.

For example, consider this modified version of the preceding example:

// Methods with differing type signatures are overloaded - not
// overridden.

```

class A {
    int i, j;

    A(int a, int b) {
        i = a;
        j = b;
    }

    // display i and j
    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}

```

// Create a subclass by extending class A.

```

class B extends A {
    int k;

    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }
}

```

```

// overload show()
void show(String msg) {
    System.out.println(msg + k);
}
}

class Override {
    public static void main(String args[]) {
        B subOb = new B(1, 2, 3);

        subOb.show("This is k: "); // this calls show() in B
        subOb.show(); // this calls show() in A
    }
}

```

The output produced by this program is shown here:

```

This is k: 3
i and j: 1 2

```

The version of **show()** in **B** takes a string parameter. This makes its type signature different from the one in **A**, which takes no parameters. Therefore, no overriding (or name hiding) takes place. Instead, the version of **show()** in **B** simply overloads the version of **show()** in **A**.

USING ABSTRACT CLASSES

There are situations in which you will want to define a superclass that declares the structure of a given abstraction without providing a complete implementation of every method. That is, sometimes you will want to create a superclass that only defines a generalized form that will be shared by all of its subclasses, leaving it to each subclass to fill in the details. Java's solution to this problem is the *abstract method*.

You can require that certain methods be overridden by subclasses by specifying the **abstract** type modifier. These methods are sometimes referred to as *subclasserresponsibility* because they have no implementation specified in the superclass.

Thus, a subclass must override them—it cannot simply use the version defined in the superclass. To declare an abstract method, use this general form:

```
abstract type name(parameter-list);
```

As you can see, no method body is present.

Any class that contains one or more abstract methods must also be declared **abstract**. To declare a class **abstract**, you simply use the **abstract** keyword in front of the **class** keyword at the beginning of the class declaration. There can be no objects of an abstract class. That is, an abstract class cannot be directly instantiated

with the **new** operator. Such objects would be useless, because an abstract class is not fully defined. Also, you cannot declare abstract constructors, or abstract static methods. Any subclass of an abstract class must either implement all of the abstract methods in the superclass.

Here is a simple example of a class with an abstract method, followed by a class which implements that method:

```
// A Simple demonstration of abstract.
abstract class A {
    abstract void callme();

    // concrete methods are still allowed in abstract classes
    void callmetoo() {
        System.out.println("This is a concrete method.");
    }
}

class B extends A {
    void callme() {
        System.out.println("B's implementation of callme.");
    }
}

class AbstractDemo {
    public static void main(String args[]) {
        B b = new B();

        b.callme();
        b.callmetoo();
    }
}
```

Notice that no objects of class **A** are declared in the program. As mentioned, it is not possible to instantiate an abstract class. One other point: class **A** implements a concrete method called **callmetoo()**. This is perfectly acceptable.

Although abstract classes cannot be used to instantiate objects, they can be used to create object references. Thus, it must be possible to create a reference to an abstract class so that it can be used to point to a subclass object.

```
// Using abstract methods and classes.
abstract class Figure {
    double dim1;
    double dim2;
```

```

Figure(double a, double b) {
    dim1 = a;
    dim2 = b;
}

// area is now an abstract method
abstract double area();
}

class Rectangle extends Figure {
    Rectangle(double a, double b) {
        super(a, b);
    }

    // override area for rectangle
    double area() {
        System.out.println("Inside Area for Rectangle.");
        return dim1 * dim2;
    }
}

class Triangle extends Figure {
    Triangle(double a, double b) {
        super(a, b);
    }

    // override area for right triangle
    double area() {
        System.out.println("Inside Area for Triangle.");
        return dim1 * dim2 / 2;
    }
}

class AbstractAreas {
    public static void main(String args[]) {
        // Figure f = new Figure(10, 10); // illegal now
        Rectangle r = new Rectangle(9, 5);
        Triangle t = new Triangle(10, 8);
        Figure figref; // this is OK, no object is created
    }
}

```

```

        figref = r;
        System.out.println("Area is " + figref.area());
        figref = t;
        System.out.println("Area is " + figref.area());
    }
}

```

As the comment inside **main()** indicates, it is no longer possible to declare objects of type **Figure**, since it is now abstract. And, all subclasses of **Figure** must override **area()**. To prove this to yourself, try creating a subclass that does not override **area()**. You will receive a compile-time error.

USING final WITH INHERITANCE

The keyword **final** has three uses. First, it can be used to create the equivalent of a named constant. The other two uses of **final** apply to inheritance.

Using final to Prevent Overriding

To disallow a method from being overridden, specify **final** as a modifier at the start of its declaration. Methods declared as **final** cannot be overridden. The following fragment illustrates **final**:

```

class A {
    final void meth() {
        System.out.println("This is a final method.");
    }
}

class B extends A {
    void meth() { // ERROR! Can't override.
        System.out.println("Illegal!");
    }
}

```

Because **meth()** is declared as **final**, it cannot be overridden in **B**. If you attempt to do so, a compile-time error will result.

Using final to Prevent Inheritance

Sometimes you will want to prevent a class from being inherited. To do this, precede the class declaration with **final**. Declaring a class as **final** implicitly declares all of its methods as **final**, too. Here is an example of a **final** class:

```

final class A {
    //...
}

// The following class is illegal.
class B extends A { // ERROR! Can't subclass A
    //...
}

```

As the comments imply, it is illegal for **B** to inherit **A** since **A** is declared as **final**.

Chapter 2

Packages and Interfaces

PACKAGES

Packages are containers for classes. They are used to keep the class namespace compartmentalized.

Defining a Package

To create a package is quite easy: simply include a **package** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package. The **package** statement defines a name space in which classes are stored. If you omit the **package** statement, the class names are put into the default package, which has no name. The default package is fine for short, sample programs. This is the general form of the **package** statement:

package *pkg*;

Here, *pkg* is the name of the package. For example, the following statement creates a package called **mypackage**:

```
package mypackage;
```

Java uses file system directories to store packages. For example, the **.class** files for any classes you declare to be part of **mypackage** must be stored in a directory called **mypackage**. Remember that case is significant, and the directory name must match the package name exactly. More than one file can include the same **package** statement. The **package** statement simply specifies to which package the classes defined in a file belong.

You can create a hierarchy of packages. To do so, simply separate each package name from the one above it by use of a period. The general form of a multileveled package statement is shown here:

```
package pkg1[.pkg2[.pkg3]];
```

A package hierarchy must be reflected in the file system of your Java development system. For example, a package declared as

```
package a.b.c;
```

needs to be stored in `a\b\c` in a Windows environment. Be sure to choose your package names carefully. You cannot rename a package without renaming the directory in which the classes are stored.

Finding Packages and CLASSPATH

Packages are mirrored by directories. This raises an important question: How does the Java run-time system know where to look for packages that you create? The answer has three parts. First, by default, the Java run-time system uses the current working directory as its starting point. Thus, if your package is in a subdirectory of the current directory, it will be found. Second, you can specify a directory path or paths by setting the **CLASSPATH** environmental variable. Third, you can use the **-classpath** option with **java** and **javac** to specify the path to your classes.

For example, consider the following package specification:

```
package mypack;
```

In order for a program to find **mypack**, the program can be executed from a directory immediately above **mypack**, or the **CLASSPATH** must be set to include the path to **mypack**, or the **-classpath** option must specify the path to **mypack** when the program is run via **java**.

When the second two options are used, the class path *must not* include **mypack**, itself. It must simply specify the *path to mypack*. For example, in a Windows environment, if the path to **mypack** is

```
C:\MyPrograms\Java\mypack
```

then the class path to **mypack** is

```
C:\MyPrograms\Java
```

A Short Package Example

```
// A simple package  
package mypack;
```

```
class Balance {  
    String name;  
    double bal;  
  
    Balance(String n, double b) {  
        name = n;  
        bal = b;  
    }  
}
```



```

void show() {
    if(bal<0)
        System.out.print("--> ");
    System.out.println(name + ": $" + bal);
}
}

class AccountBalance {
    public static void main(String args[]) {
        Balance current[] = new Balance[3];
        current[0] = new Balance("K. J. Fielding", 123.23);
        current[1] = new Balance("Will Tell", 157.02);
        current[2] = new Balance("Tom Jackson", -12.33);

        for(int i=0; i<3; i++) current[i].show();
    }
}

```

Call this file **AccountBalance.java** and put it in a directory called **mypack**. Next, compile the file. Make sure that the resulting **.class** file is also in the **mypack** directory. Then, try executing the **AccountBalance** class, using the following command line:

```
java mypack.AccountBalance
```

Remember, you will need to be in the directory above **mypack** when you execute this command.

As explained, **AccountBalance** is now part of the package **mypack**. This means that it cannot be executed by itself. That is, you cannot use this command line:

```
java AccountBalance
```

AccountBalance must be qualified with its package name.

ACCESS PROTECTION

Classes and packages are both means of encapsulating and containing the name space and scope of variables and methods. Packages act as containers for classes and other subordinate packages. Classes act as containers for data and code. The class is Java's smallest unit of abstraction. Java addresses four categories of visibility for class members:

- Subclasses in the same package
- Non-subclasses in the same package
- Subclasses in different packages

- Classes that are neither in the same package nor subclasses

The three access modifiers, **private**, **public**, and **protected**, provide a variety of ways to produce the many levels of access required by these categories.

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Anything declared **public** can be accessed from different classes and different packages. Anything declared **private** cannot be seen outside of its class.

When a member does not have an explicit access specification, it is visible to subclasses as well as to other classes in the same package. This is the default access. If you want to allow an element to be seen outside your current package, but only to classes that subclass your class directly, then declare that element **protected**.

IMPORTING PACKAGES

Java includes the **import** statement to bring certain classes, or entire packages, into visibility. Once imported, a class can be referred to directly, using only its name.

The **import** statement is a convenience to the programmer and is not technically needed to write a complete Java program.

In a Java source file, **import** statements occur immediately following the **package** statement (if it exists) and before any class definitions. This is the general form of the **import** statement:

```
import pkg1 [.pkg2].(classname | *);
```

Here, *pkg1* is the name of a top-level package, and *pkg2* is the name of a subordinate package inside the outer package separated by a dot (.). There is no practical limit on the depth of a package hierarchy. Finally, you specify either an explicit *classname* or a star (*), which indicates that the Java compiler should import the entire package. This code fragment shows both forms in use:

```
import java.util.Date;
```

```
import java.io.*;
```

The **import** statement is optional. Any place you use a class name, you can use its *fully qualified name*, which includes its full package hierarchy. For example, this fragment uses an import statement:

```
import java.util.*;
class MyDate extends Date {
}
```

The same example without the **import** statement looks like this:

```
class MyDate extends java.util.Date {  
}
```

In this version, **Date** is fully-qualified.

```
package mypack;
```

```
/* Now, the Balance class, its constructor, and its  
   show() method are public. This means that they can  
   be used by non-subclass code outside their package.  
*/
```

```
public class Balance {  
    String name;  
    double bal;  
  
    public Balance(String n, double b) {  
        name = n;  
        bal = b;  
    }  
  
    public void show() {  
        if(bal<0)  
            System.out.print("--> ");  
        System.out.println(name + ": $" + bal);  
    }  
}
```

As you can see, the **Balance** class is now **public**. Also, its constructor and its **show()** method are **public**, too. This means that they can be accessed by any type of code outside the **mypack** package. For example, here **TestBalance** imports **mypack** and is then able to make use of the **Balance** class:

```
import mypack.*;

class TestBalance {
    public static void main(String args[]) {

        /* Because Balance is public, you may use Balance
           class and call its constructor. */
        Balance test = new Balance("J. J. Jaspers", 99.88);

        test.show(); // you may also call show()
    }
}
```

INTERFACES

Using the keyword **interface**, you can fully abstract a class interface from its implementation. Interfaces are syntactically similar to classes, but they lack instance variables, and their methods are declared without any body. Once it is defined, any number of classes can implement an **interface**. Also, one class can implement any number of interfaces.

To implement an interface, a class must provide the complete set of methods required by the interface. By providing the **interface** keyword, Java allows you to fully utilize the “one interface, multiple methods” aspect of polymorphism.

Defining an Interface

An interface is defined much like a class. This is a simplified general form of an interface:

```
access interface name {
    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);

    type final-varname1 = value;
    type final-varname2 = value;
    //...
    return-type method-nameN(parameter-list);
    type final-varnameN = value;
}
```

When no access modifier is included, then default access results, and the interface is only available to other members of the package in which it is declared. When it

is declared as **public**, the interface can be used by code outside its package. In this case, the interface must be the only public interface declared in the file, and the file must have the same name as the interface. *name* is the name of the interface, and can be any valid identifier. Notice that the methods that are declared have no bodies. They end with a semicolon after the parameter list.

As the general form shows, variables can be declared inside interface declarations. They are implicitly **final** and **static**, meaning they cannot be changed by the implementing class. They must also be initialized. All methods and variables are implicitly **public**.

Here is an example of an interface definition. It declares a simple interface that contains one method called **callback()** that takes a single integer parameter.

```
interface Callback {  
    void callback(int param);  
}
```

Implementing Interfaces

Once an **interface** has been defined, one or more classes can implement that interface. To implement an interface, include the **implements** clause in a class definition, and then create the methods required by the interface. The general form of a class that includes the **implements** clause looks like this:

```
class classname [extends superclass] [implements interface [,interface...]] {  
    // class-body  
}
```

If a class implements more than one interface, the interfaces are separated with a comma. The methods that implement an interface must be declared **public**. Also, the type signature of the implementing method must match exactly the type signature specified in the **interface** definition.

Here is a small example class that implements the **Callback** interface shown earlier:

```
class Client implements Callback {  
    // Implement Callback's interface  
    public void callback(int p) {  
  
        System.out.println("callback called with " + p);  
    }  
}
```

Notice that **callback()** is declared using the **public** access modifier.

```

class Client implements Callback {
    // Implement Callback's interface
    public void callback(int p) {
        System.out.println("callback called with " + p);
    }

    void nonIfaceMeth() {
        System.out.println("Classes that implement interfaces " +
                           "may also define other members, too.");
    }
}

```

Chapter 3

Exception Handling

An *exception* is an abnormal condition that arises in a code sequence at run time. In other words, an exception is a run-time error. In computer languages that do not support exception handling, errors must be checked and handled manually—typically through the use of error codes, and so on. This approach is as cumbersome as it is troublesome. Java's exception handling avoids these problems and, in the process, brings run-time error management into the object-oriented world.

EXCEPTION-HANDLING FUNDAMENTALS

A Java exception is an object that describes an exceptional (that is, error) condition that has occurred in a piece of code. When an exceptional condition arises, an object representing that exception is created and *thrown* in the method that caused the error. That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is *caught* and processed. Exceptions can be generated by the Java run-time system, or they can be manually generated by your code.

Java exception handling is managed via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**. Briefly, here is how they work. Program statements that you want to monitor for exceptions are contained within a **try** block. If an exception occurs within the **try** block, it is thrown. Your code can catch this exception (using **catch**) and handle it in some rational manner. System-generated exceptions are automatically thrown by the Java run-time system. To manually throw an exception, use the keyword **throw**. Any exception that is thrown out of a method must be specified as such by a **throws** clause. Any code that absolutely must be executed after a **try** block completes is put in a **finally** block.

This is the general form of an exception-handling block:

```

try {
    // block of code to monitor for errors
}

catch (ExceptionType1 exOb) {
    // exception handler for ExceptionType1
}

catch (ExceptionType2 exOb) {
    // exception handler for ExceptionType2
}
// ...
finally {
    // block of code to be executed after try block ends
}

```

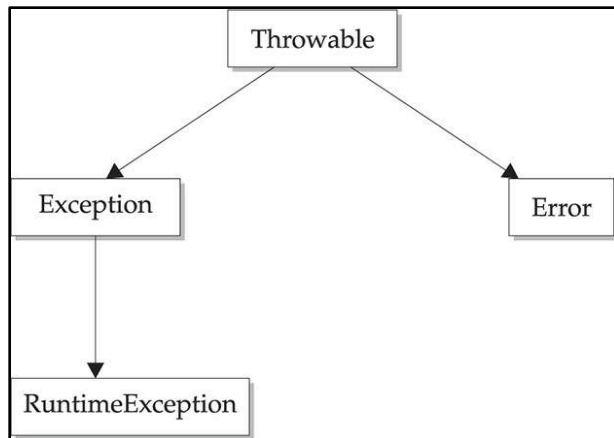
Here, *ExceptionType* is the type of exception that has occurred.

EXCEPTION TYPES

All exception types are subclasses of the built-in class **Throwable**.

Thus, **Throwable** is at the top of the exception class hierarchy. Immediately below **Throwable** are two subclasses. One branch is headed by **Exception**. This class is used for exceptional conditions that user programs should catch. There is an important subclass of **Exception**, called **RuntimeException**. Exceptions of this type are automatically defined for the programs that you write and include things such as division by zero and invalid array indexing. The other branch is topped by **Error**, which defines exceptions that are not expected to be caught under normal circumstances by your program. Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself. Stack overflow is an example of such an error.

The top-level exception hierarchy is shown here:



UNCAUGHT EXCEPTIONS

This small program includes an expression that intentionally causes a divide-by-zero error:

```

class Exc0 {
    public static void main(String args[]) {
        int d = 0;
        int a = 42 / d;
    }
}
  
```

When the Java run-time system detects the attempt to divide by zero, it constructs a new exception object and then *throws* this exception. This causes the execution of **Exc0** to stop, because once an exception has been thrown, it must be *caught* by an exception handler and dealt with immediately. In this example, we haven't supplied any exception handlers of our own, so the exception is caught by the default handler provided by the Java run-time system. Any exception that is not caught by your program will ultimately be processed by the default handler. The default handler displays a string describing the exception, prints a stack trace from the point at which the exception occurred, and terminates the program.

Here is the exception generated when this example is executed:

```

java.lang.ArithmeticException: / by zero
    at Exc0.main(Exc0.java:4)
  
```

Notice how the class name, **Exc0**; the method name, **main**; the filename, **Exc0.java**; and the line number, **4**, are all included in the simple stack trace. Also, notice that the type of exception thrown is a subclass of **Exception** called **ArithmeticException**, which more specifically describes what type of error happened.

Here is another version of the preceding program that introduces the same error but in a method separate from **main()**:


```

class Exc1 {
    static void subroutine() {
        int d = 0;
        int a = 10 / d;
    }
    public static void main(String args[]) {
        Exc1.subroutine();
    }
}

```

The resulting stack trace from the default exception handler shows how the entire call stack is displayed:

```

java.lang.ArithmeticException: / by zero
    at Exc1.subroutine(Exc1.java:4)
    at Exc1.main(Exc1.java:7)

```

As you can see, the bottom of the stack is **main**'s line 7, which is the call to **subroutine()**, which caused the exception at line 4. The call stack is quite useful for debugging, because it pinpoints the precise sequence of steps that led to the error.

USING try AND catch

Although the default exception handler provided by the Java run-time system is useful for debugging, you will usually want to handle an exception yourself. Doing so provides two benefits. First, it allows you to fix the error. Second, it prevents the program from automatically terminating.

To guard against and handle a run-time error, simply enclose the code that you want to monitor inside a **try** block. Immediately following the **try** block, include a **catch** clause that specifies the exception type that you wish to catch.

```

class Exc2 {
    public static void main(String args[]) {
        int d, a;

        try { // monitor a block of code.
            d = 0;
            a = 42 / d;
            System.out.println("This will not be printed.");
        } catch (ArithmeticException e) { // catch divide-by-zero error
            System.out.println("Division by zero.");
        }
    }
}

```

```

        System.out.println("After catch statement.");
    }
}

```

This program generates the following output:

Division by zero.

After catch statement.

Notice that the call to **println()** inside the **try** block is never executed. Once an exception is thrown, program control transfers out of the **try** block into the **catch** block. The execution never “returns” to the **try** block from a **catch**. Thus, the line “This will not be printed.” is not displayed. Once the **catch** statement has executed, program control continues with the next line in the program following the entire **try / catch** mechanism.

A **try** and its **catch** statement form a unit. A **catch** statement cannot catch an exception thrown by another **try** statement. The statements that are protected by **try** must be surrounded by curly braces. You cannot use **try** on a single statement.

Displaying a Description of an Exception

Throwable overrides the **toString()** method (defined by **Object**) so that it returns a string containing a description of the exception. You can display this description in a **println()** statement by simply passing the exception as an argument. For example, the **catch** block in the preceding program can be rewritten like this:

```

catch (ArithmeticException e) {
    System.out.println("Exception: " + e);
    a = 0; // set a to zero and continue
}

```

When this version is substituted in the program, and the program is run, each divide-by-zero error displays the following message:

Exception: java.lang.ArithmeticException: / by zero

MULTIPLE CATCH CLAUSES

In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, you can specify two or more **catch** clauses, each catching a different type of exception. When an exception is thrown, each **catch** statement is inspected in order, and the first one whose type matches that of the exception is executed. After one **catch** statement executes, the others are bypassed, and execution continues after the **try / catch** block. The following example traps two different exception types:

```
// Demonstrate multiple catch statements.
class MultipleCatches {
    public static void main(String args[]) {
        try {
            int a = args.length;
            System.out.println("a = " + a);
            int b = 42 / a;
            int c[] = { 1 };
            c[42] = 99;
        } catch(ArithmeticException e) {
            System.out.println("Divide by 0: " + e);
        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index oob: " + e);
        }
        System.out.println("After try/catch blocks.");
    }
}
```

This program will cause a division-by-zero exception if it is started with no command-line arguments, since **a** will equal zero. It will survive the division if you provide a command-line argument, setting **a** to something larger than zero. But it will cause an **ArrayIndexOutOfBoundsException**, since the **int** array **c** has a length of 1, yet the program attempts to assign a value to **c[42]**.

Here is the output generated by running it both ways:

```
C:\>java MultipleCatches
```

```
a = 0
```

```
Divide by 0: java.lang.ArithmeticException: / by zero
```

```
After try/catch blocks.
```

```
C:\>java MultipleCatches TestArg
```

```
a = 1
```

```
Array index oob: java.lang.ArrayIndexOutOfBoundsException:42
```

```
After try/catch blocks.
```

When you use multiple **catch** statements, it is important to remember that exception subclasses must come before any of their superclasses. This is because a **catch** statement that uses a superclass will catch exceptions of that type plus any of its subclasses. Thus, a subclass would never be reached if it came after its superclass. Further, in Java, unreachable code is an error. For example, consider the following program:

```
/* This program contains an error.
```

```
    A subclass must come before its superclass in
    a series of catch statements. If not,
    unreachable code will be created and a
    compile-time error will result.
```

```
*/
class SuperSubCatch {
    public static void main(String args[]) {
        try {
            int a = 0;
            int b = 42 / a;
        } catch(Exception e) {
            System.out.println("Generic Exception catch.");
        }
        /* This catch is never reached because
           ArithmeticException is a subclass of Exception. */
        catch(ArithmeticException e) { // ERROR - unreachable
            System.out.println("This is never reached.");
        }
    }
}
```

If you try to compile this program, you will receive an error message stating that the second **catch** statement is unreachable because the exception has already been caught. Since **ArithmeticException** is a subclass of **Exception**, the first **catch** statement will handle all **Exception**-based errors, including **ArithmeticException**.

This means that the second **catch** statement will never execute. To fix the problem, reverse the order of the **catch** statements.

USING finally

finally creates a block of code that will be executed after a **try /catch** block has completed and before the code following the **try/catch** block. The **finally** block will execute whether or not an exception is thrown. If an exception is thrown, the **finally** block will execute even if no **catch** statement matches the exception. The **finally** clause is optional.

However, each **try** statement requires at least one **catch** or a **finally** clause. Here is an example program that shows three methods that exit in various ways, none without executing their **finally** clauses:

```

// Demonstrate finally.
class FinallyDemo {
    // Throw an exception out of the method.
    static void procA() {
        try {
            System.out.println("inside procA");
            throw new RuntimeException("demo");
        } finally {
            System.out.println("procA's finally");
        }
    }

    // Return from within a try block.
    static void procB() {
        try {
            System.out.println("inside procB");
            return;
        } finally {
            System.out.println("procB's finally");
        }
    }

    // Execute a try block normally.
    static void procC() {
        try {
            System.out.println("inside procC");
        } finally {
            System.out.println("procC's finally");
        }
    }

    public static void main(String args[]) {
        try {
            procA();
        } catch (Exception e) {
            System.out.println("Exception caught");
        }

        procB();
        procC();
    }
}

```

In this example, **procA()** prematurely breaks out of the **try** by throwing an exception. The **finally** clause is executed on the way out. **procB()**'s **try** statement is exited via a **return** statement. The **finally** clause is executed before **procB()** returns. In **procC()**, the **try** statement executes normally, without error. However, the **finally** block is still executed.

Here is the output generated by the preceding program:

```
inside procA
procA's finally
Exception caught
inside procB
procB's finally
inside procC
procC's finally
```

JAVA'S BUILT-IN EXCEPTIONS

Inside the standard package **java.lang**, Java defines several exception classes. Some exceptions need not be included in any method's **throws** list. In the language of Java, these are called *unchecked exceptions* because the compiler does not check to see if a method handles or throws these exceptions. The unchecked exceptions defined in **java.lang** are listed in [below table](#). [The second table](#) lists those exceptions defined by **java.lang** that must be included in a method's **throws** list if that method can generate one of these exceptions and does not handle it itself. These are called *checked exceptions*.

Java's Unchecked RuntimeException Subclasses Defined in java.lang

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalCallerException	A method cannot be legally executed by the calling code. (Added by JDK 9.)
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.

IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
LayerInstantiationException	A module layer cannot be created. (Added by JDK 9.)
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Java's Checked Exceptions Defined in java.lang

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions.

Chapter 4 Multithreaded Programming

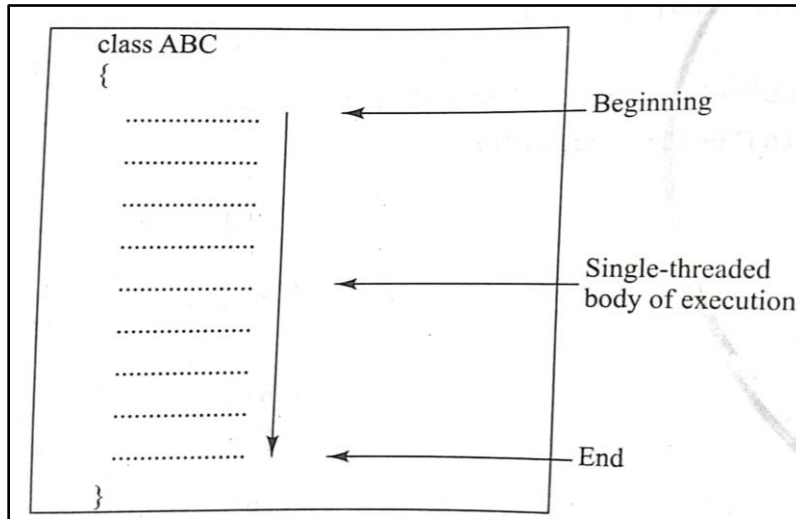
INTRODUCTION

Multithreading is a conceptual programming paradigm where a program or process is divided into two or more subprograms, which can be implemented at the same time in parallel. For example, one subprogram can display an animation on the screen while another may build the next animation to be displayed. This is something similar to dividing a task into subtasks and assigning them to different people for execution independently and simultaneously.

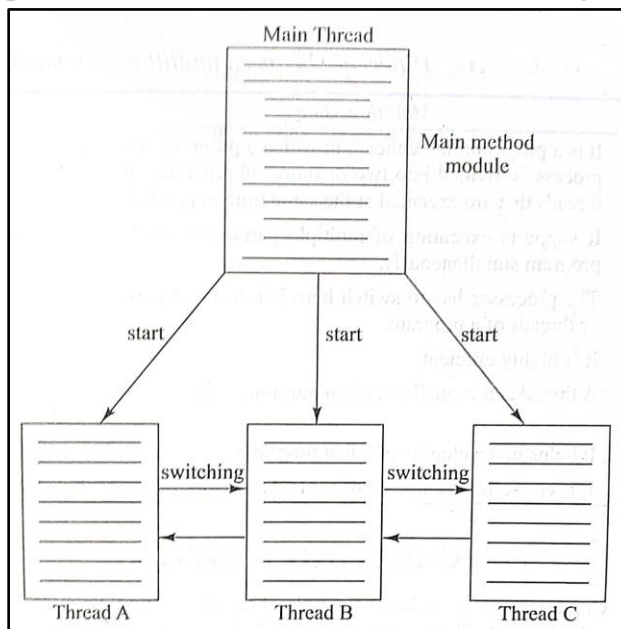
In most of our computers, we have only a single processor and therefore, the processor is doing only one thing at a time. Java programs have a single sequential

flow of control. The program begins, runs through a sequence of executions, and finally ends.

A thread is similar to a program that has a single flow of control. It has a beginning, a body, and an end, and executes commands sequentially. Every program will have at least one thread and which is shown in below figure.



A unique property of Java is its support for multithreading. Java enables us to use multiple flows of control in developing programs. Each flow of control may be thought of as a separate tiny program or module known as a thread that runs in parallel to others as shown in below figure



A program that contains multiple flows of control is known as multithreaded program. The above figure illustrates a Java program with 3 threads, one main

thread and three others. The main thread is the main method module, which is designed to create and start the other 3 threads, namely A, B, and C.

Once initiated by the main thread, the threads A, B and C run concurrently and share the resources jointly. It is like people living in joint families and sharing resources among all of them. The ability of a language to support multithreads is referred to as concurrency. Since threads in Java are subprograms of a main application program and share the same memory space, they are known as lightweight threads or lightweight processes.

It is important to remember that ‘threads running in parallel’ does not really mean that they actually run at the same time. Since all the threads are running on a single processor, the flow of execution is shared between the threads. The Java interpreter handles the switching of controls between the threads in such a way that it appears they are running concurrently.

Multithreading is a powerful programming tool that makes Java different from other programming languages. Multithreading is useful in a number of ways. It enables programmers to do multiple things at one time. They can divide a long program into threads and execute them in parallel. It will improve the speed of our programs.

The difference between multithreading and multitasking is given below:

Multithreading	Multitasking
It is a programming concept in which a program or process is divided into 2 or more subprograms or threads that are executed at the same time in parallel	It is an operating system concept in which multiple tasks are performed simultaneously
It supports execution of multiple parts of a single program simultaneously	It supports execution of multiple programs simultaneously
The processor has to switch between different parts or threads of a program	The processor has to switch between different programs or processors
It is highly efficient	It is less efficient in comparison to multithreading
A thread is the smallest unit in multithreading	A program or processor is the smallest unit in a multitasking environment
It helps in developing efficient programs	It helps in developing efficient operating systems
It is cost effective	It is expensive

CREATING THREADS

Creating threads in Java is simple. Threads are implemented in the form of objects that contain a method called **run()**. The **run()** method is the heart and soul of any thread. It makes up the entire body of a thread and is the only method in which the threads behaviour can be implemented. A typical **run()** would appear as follows:

```
public void run()
{
    .....
    ..... (Statements for implementing thread)
    .....
}
```

The **run()** method should be invoked by an object of the concerned thread. This can be achieved by creating the thread and initiating it with the help of another thread method called **start()**.

A new thread can be created in 2 ways:

1. **By creating a thread class:** Define a class that extends **Thread** class and override its **run()** method with the code required by the thread
2. **By converting a class to a thread:** Define a class that implements **Runnable** interface. The **Runnable** interface has only one method **run()** that is to be defined in the method with the code to be executed by the thread.

EXTENDING THE THREAD CLASS

We can make our class runnable as thread by extending the class **java.lang.thread**. This gives us access to all the thread methods directly. It includes the following steps:

1. Declare the class as extending the **Thread** class
2. Implement the **run()** method that is responsible for executing the sequence of code that the thread will execute.
3. Create a thread object and call the **start()** method to initiate the thread execution

Declaring the Class

The **Thread** class can be extended as follows:

```
class MyThread extends Thread
{
    .....
    .....
    .....
}
```

Now we have a new type of thread **MyThread**

Implementing the run() Method

The **run()** method has been inherited by the class **MyThread**. The basic implementation of **run()** method will look like this:

```
public void run()
{
    .....
    .....
    .....
}
```

When we start the new thread, Java calls the threads **run()** method, so it is the **run()** where all the action takes place.

Starting New Thread

To actually create and run an instance of a thread class we must write the following:

```
MyThread aThread=new MyThread();
aThread.start(); //invokes run() method
```

The first line instantiate a new object of class **MyThread**. This statement just creates the object. The thread is in a newborn state.

The second line calls the **start()** method causing the thread to move into the runnable state. Then, the Java runtime will schedule the thread to run by invoking its **run()** method. Now, the thread is said to be in ready the running state.

An Example of Using the Thread Class

The below program illustrates the use of **Thread** class for creating and running threads in an application. The program creates 3 threads A, B and C for undertaking 3 different tasks. The **main** method in the **ThreadTest** class also constitutes another thread which we may call the **main thread**.

The main thread dies at the end of its **main** method. However, before it dies, it creates and starts all the 3 threads A, B and C.

Note the statement: `new A().start();` in the main thread. This is just a compact way of starting a thread. This is equivalent to:

```
A threadA=new A();
threadA.start();
```

Immediately, after the thread A is started, there will be two threads running in the program: the main thread and the thread A. The **start()** method returns back to the main thread immediately after invoking the **run()** method, thus allowing the main thread to start the thread B.

Similarly, it starts C thread. By the time the main thread has reached the end of its main method, there are a total of four separate threads running in parallel.

```
class A extends Thread
{
    public void run()
    {
        for (int i=1; i<=5; i++)
        {
            System.out.println("\t From ThreadA : i = "+i);
        }
        System.out.println("Exit from A ");
    }
}

class B extends Thread
{
    public void run()
    {
        for (int j=1; j<=5; j++)
        {
            System.out.println("\tFrom Thread B :j =" +j);
        }
        System.out.println("Exit from B ");
    }
}
```

OUTPUT

```
From ThreadA : i = 1
From Thread B :j =1
From ThreadA : i = 2
From Thread B :j =2
From ThreadA : i = 3
From Thread B :j =3
From ThreadA : i = 4
From Thread B :j =4
From ThreadA : i = 5
From Thread B :j =5
From Thread C: k = 1
Exit from A
    From Thread C: k = 2
Exit from B
    From Thread C: k = 3
    From Thread C: k = 4
    From Thread C: k = 5
Exit from C
```

```

class C extends Thread
{
public void run()
{
for (int k=1; k<=5; k++)
{
System.out.println("\tFrom Thread C: k = " + k);
}
System.out.println("Exit from C ");
}
}
class ThreadTest
{
public static void main(String args[])
{
new A().start();
new B().start();
new C().start();
}
}

```

We have simply initiated three new threads and started them. They are running concurrently on their own. Note that the output from the threads is not sequential. They do not follow any specific order. They are running independently of one another and each executes whenever it has a chance.

STOPPING AND BLOCKING A THREAD

Stopping a Thread

Whenever we want to stop a thread from running further, we may do so by calling its **stop()** method, like: `aThread.stop();`

This statement causes the thread to move to the **dead** state. A thread will also move to the dead state automatically when it reaches the end of its method. The **stop()** method may be used when the premature death of a thread is desired.

Blocking a Thread

A thread can also be temporarily suspended or blocked by using either of the following thread methods:

`sleep()` // blocked for a specified time

`suspend()` // blocked until further orders

`wait()` // blocked until certain condition occurs

These methods cause the thread to go into the **blocked** (or **not-runnable**) state.

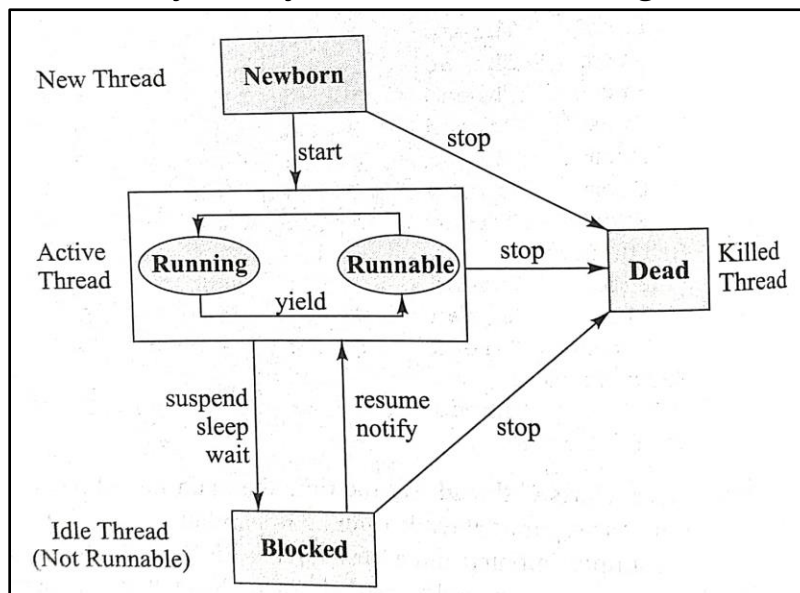
The thread will return to the runnable state when the specified time is elapsed in the case of **sleep()**, the **resume()** method is invoked in the case of **suspend()**, and the **notify()** method is called in the case of **wait()**.

LIFE CYCLE OF A THREAD

During the life time of a thread, there are many states it can enter. They include:

1. Newborn state
2. Runnable state
3. Running state
4. Blocked state
5. Dead state

A thread is always in one of these five states. It can move from one state to another via a variety of ways as shown in below figure

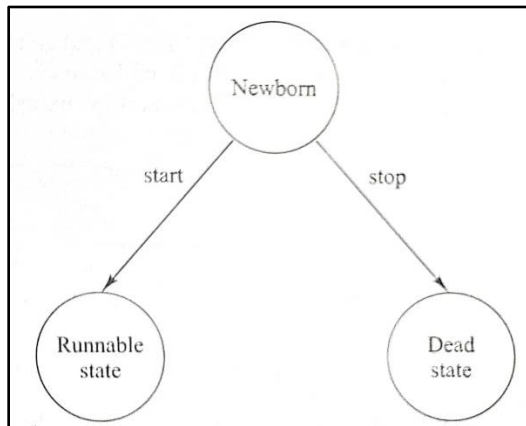


Newborn State

When we create a thread object, the thread is born and is said to be in newborn state. The thread is not yet scheduled for running. At this state, we can do only one of the following things with it:

- Schedule it for running using **start()**
- Kill it using **stop()** method.

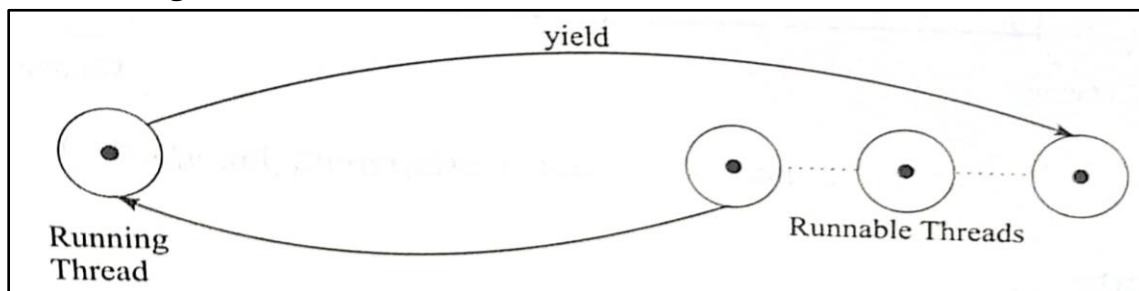
If scheduled, it moves to the runnable state as shown in below figure. If we attempt to use any other method at this stage, an exception will be thrown.



Runnable State

The runnable state means that the thread is ready for execution and is waiting for the availability of the processor. If all threads have equal priority, then they are given time slots for execution in round robin fashion, i.e., first-come, first-serve manner. The thread that relinquishes control joins the queue at the end and again waits for its turn. This process of assigning time to threads is known as **time-slicing**.

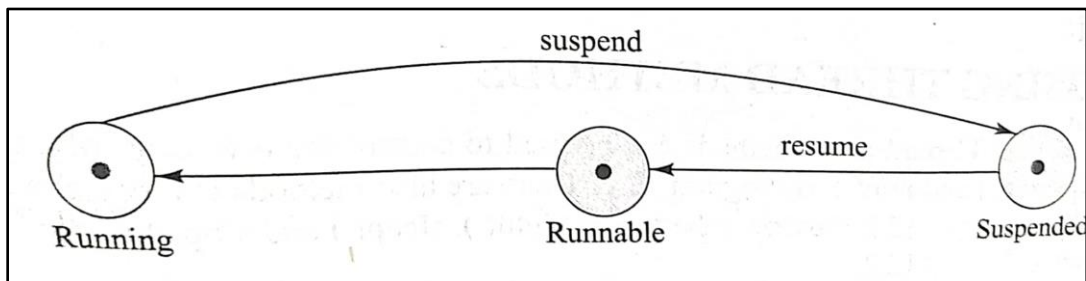
However, if we want a thread to relinquish control to another thread to equal priority before its turn comes, we can do so by using the **yield()** method as shown in below figure



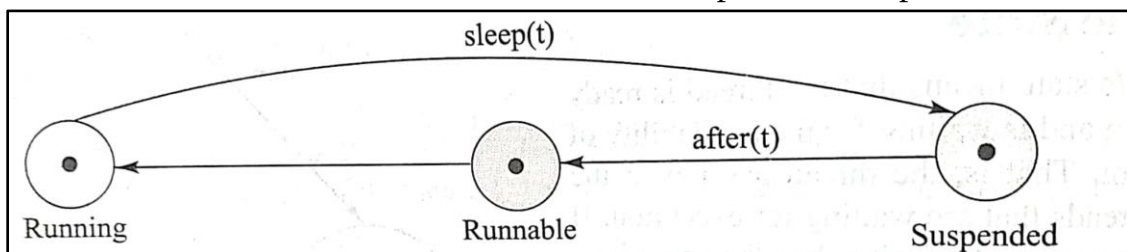
Running State

Running means that the processor has given its time to the thread for its execution. The thread runs until it relinquishes control on its own or it is preempted by a higher priority thread. A running thread may relinquish its control in one of the following situations.

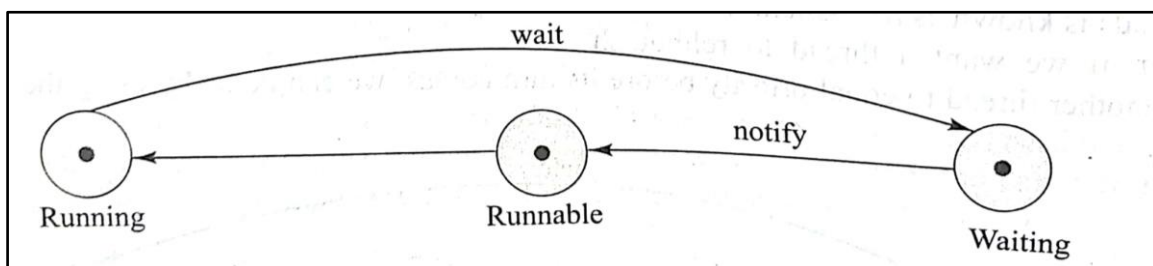
1. It has been suspended using **suspend()** method. A suspended thread can be revived by using the **resume()** method. This approach is useful when we want to suspend a thread for some time due to certain reason, but do not want to kill it.



2. It has been made to sleep. We can put a thread to sleep for a specified time period using the method **sleep(time)** where time is in milliseconds. This means that the thread is out of the queue during this time period. The thread re-enters the runnable state as soon as this time period is elapsed.



3. It has been told to wait until some event occurs. This is done using the **wait()** method. The thread can be scheduled to run again using the **notify()** method.



Blocked State

A thread is said to be blocked when it is prevented from entering into the runnable state and subsequently the running state. This happens when the thread is suspended, sleeping, or waiting in order to satisfy certain requirements. A blocked thread is considered "not runnable" but not dead and therefore fully qualified to run again.

Dead State

Every thread has a life cycle. A running thread ends its life when it has completed executing its **run()** method. It is a natural death. However, we can kill it by sending the stop message to it at any state thus causing a premature death to it. A thread can be killed as soon it is born, or while it is running, or even when it is in "not runnable" (blocked) condition.

USING THREAD METHODS

We used the methods start() and run() in above program. There are also methods that can move a thread from one state to another. The below program illustrates the use of yield(), sleep() and stop() methods.

```
class A extends Thread
{
public void run()
{
for (int i = 1; i<=5; i++)
{
if (i==1)
yield();
System.out.println("\tFrom Thread A:i="+i);
}
System.out.println("exit from A");
}
}
class B extends Thread
{
public void run()
{
```

```

for (int j=1; j<=5; j++)
{
System.out.println("\tFrom Thread B : j="+j);
if(j==3)
stop();
}
System.out.println("Exit from B ");
}
}
class C extends Thread
{
public void run()
{
for (int k=1; k<=5; k++)
{
System.out.println("\tFrom Thread C: k =" +k);
if (k==1)
try
{
sleep (1000);
}
catch(Exception e)
{
}
System.out.println("Exit from C ");
}
}
}
class ThreadMethods
{
public static void main(String args[])
{
A threadA=new A();
B threadB=new B();

```

OUTPUT

```

Start thread A
start thread B
    From Thread A:i=1
Start thread C
    From Thread B : j=1
    From Thread A:i=2
    From Thread C: k =1
    From Thread B : j=2
End of main thread
    From Thread B : j=3
    From Thread A:i=3
    From Thread A:i=4
    From Thread A:i=5
exit from A
Exit from C
    From Thread C: k =2
Exit from C
    From Thread C: k =3
Exit from C
    From Thread C: k =4
Exit from C
    From Thread C: k =5
Exit from C

```

```

C threadC=new C();
System.out.println("Start thread A");
threadA.start();
System.out.println("start thread B");
threadB.start();
System.out.println("Start thread C");
threadC.start();
System.out.println("End of main thread");
}
}

```

The above program uses the **yield()** method in thread A at the iteration $i = 1$. Therefore, the thread A, although started first, has relinquished its control to the thread B. The **stop()** method in thread B has killed it after implementing the for loop only three times. Note that it has not reached end of **run()** method. The threadC started sleeping after executing the for loop only once. When it woke up (after 1000 milliseconds), the other two threads have already completed their runs and therefore was running alone. The main thread died much earlier than the other three threads.

IMPLEMENTING THE 'RUNNABLE' INTERFACE

We stated earlier that we can create threads in two ways: one by using the extended **Thread** class and another by implementing the **Runnable** interface.

The **Runnable** interface declares the **run()** method that is required for implementing threads in our programs. To do this, we must perform the steps listed below:

1. Declare the class as implementing the **Runnable** interface.
2. Implement the **run()** method.
3. Create a thread by defining an object that is instantiated from this "runnable" class as the target of the thread.
4. Call the thread's **start()** method to run the thread.

The below program illustrates the implementation of the above steps.

```

class X implements Runnable // Step 1
{
public void run()// Step 2

```

```

{
for (int i = 1; i<10; i++)
{
System.out.println("\tThreadX" +i);
}
System.out.println("End of ThreadX");
}
}
class RunnableTest
{
public static void main(String args[])
{
X runnable=new X();
Thread threadX=new Thread( runnable);// Step 3
threadX. start(); // Step 4
System.out.println("End of main Thread");
}
}

```

OUTPUT

```

End of main Thread
    ThreadX1
    ThreadX2
    ThreadX3
    ThreadX4
    ThreadX5
    ThreadX6
    ThreadX7
    ThreadX8
    ThreadX9
End of ThreadX

```